# Mandelbrot com MPI: mestre/escravo

Max Weiss Breuel e George Rother | Trabalho 2 | 02 nov 2026
github.com/George-Rot/FractalOpenMpi

## Modelagem e protocolo

Uma tarefa é uma linha independente da imagem (src/mandel.h:24). O mestre guarda a próxima linha do saco e entrega blocos de até **grain=4** (src/mandel_mpi.c:39-68). Cada escravo envia **TAG_REQUEST** inicialmente; o resultado **TAG_RESULT** traz posição, quantidade e pixels, e já pede novo bloco. O mestre usa **MPI_ANY_SOURCE**, recoloca os pixels na posição original e responde com **TAG_WORK** ou **TAG_STOP**. As estatísticas são reunidas por **MPI_Gather** após o término (src/mandel_mpi.c:80-88, 102-127).

## Ambiente e medição

Jobs exclusivos nos nós **atlantica03/04**; o sequencial rodou no 03. Seu lscpu registrou Xeon E5520, **2 soquetes × 4 núcleos × 2 threads** (16 CPUs lógicas), divergindo dos 16 núcleos físicos por nó descritos no enunciado. O Slurm reservou 16 tarefas por nó; srun distribuiu ranks ciclicamente, incluindo o mestre em P. P=32 usa HT. As versões usam **-O3 -std=c11 -Wall -Wextra** (Makefile:6). Entrada base: 2000 × 4000 pixels, 2000 iterações máximas; na fraca, a altura vira 4000P. Foram 10 repetições por configuração.

O tempo paralelo usa **MPI_Wtime** no mestre, após a barreira, da distribuição até o último resultado (src/mandel_mpi.c:44-72). O sequencial mede apenas o laço de linhas (src/mandel_seq.c:32-35). A mediana T1 foi **15,126 s** [15,122; 15,128]. Alocação, checksum e reunião de estatísticas ficam fora da região medida.

## Equivalência e escopo

Todos os checksums paralelos coincidiram com os sequenciais da mesma dimensão; na base, **10899745284484**. A fraca elevou as iterações de 2,723 para 87,121 bilhões em P=32, razão **31,99×**, próxima dos 32× planejados. O conjunto entregue contém logs brutos dos jobs 41967/41968 e as tabelas reproduzíveis.

**Tabela 1 - Escalabilidade forte (s; S=T1/Tp, E=S/P).**

| P | med | mín | máx | S | E |
|---|---|---|---|---|---|
| 2 | 15.631 | 15.608 | 15.643 | 0.97 | 0.484 |
| 4 | 5.145 | 5.136 | 5.149 | 2.94 | 0.735 |
| 8 | 2.222 | 2.211 | 2.232 | 6.81 | 0.851 |
| 16 | 1.094 | 1.093 | 1.095 | 13.83 | 0.864 |
| 32 | 0.621 | 0.620 | 0.623 | 24.37 | 0.762 |

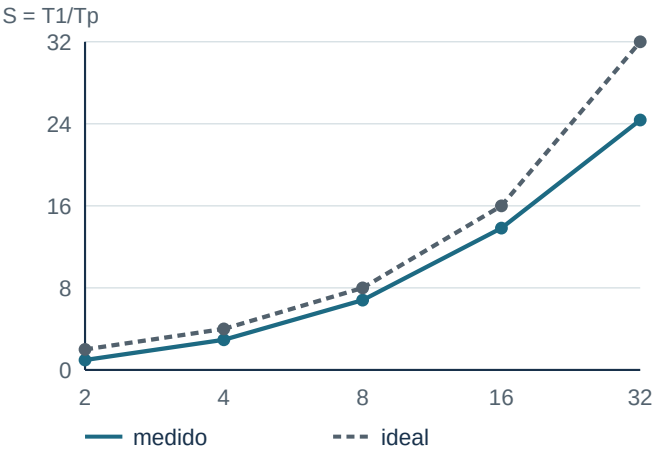


**Figura 1 - Speed-up forte e referência ideal.**

Na Tabela 1 e Figura 1, P=2 fica abaixo de 1 porque só um rank calcula e a comunicação soma custo. O ganho cresce até **24,37×** em P=32: não há platô medido. A eficiência cai de **0,864** (P=16) para **0,762** (P=32); nesse ponto cada nó passa de 8 para 16 ranks e usa a segunda thread por núcleo. Isso é compatível com retorno decrescente do HT, sem isolá-lo de outros custos.

**Tabela 2 - Escalabilidade fraca (iterações em bilhões; tempo em s; E=T1/Tp).**

| P | linhas | it. G | med | mín | máx | E |
|---|---|---|---|---|---|---|
| 2 | 8k | 5.45 | 31.255 | 31.174 | 31.303 | 0.484 |
| 4 | 16k | 10.89 | 20.539 | 20.516 | 20.557 | 0.736 |
| 8 | 32k | 21.78 | 17.742 | 17.671 | 17.782 | 0.853 |
| 16 | 64k | 43.56 | 17.450 | 17.432 | 17.464 | 0.867 |
| 32 | 128k | 87.12 | 19.657 | 19.639 | 19.677 | 0.769 |

# Mandelbrot com MPI: mestre/escravo

Max Weiss Breuel e George Rother | Trabalho 2 | 02 nov 2026
github.com/George-Rot/FractalOpenMpi

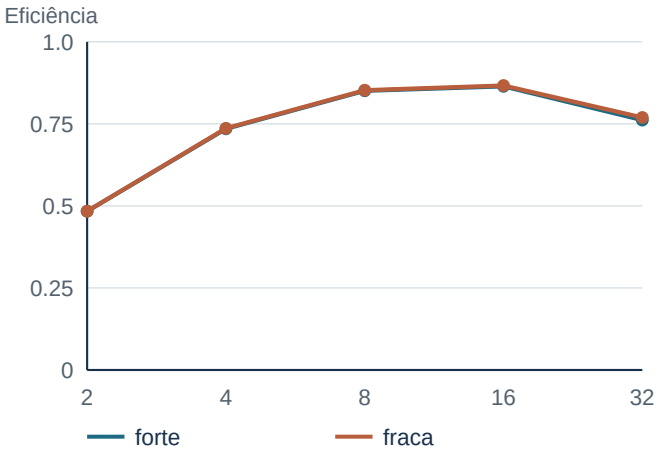


**Figura 2 - Eficiências forte e fraca por P.**

A Tabela 2 e a Figura 2 mostram que E fraca sobe de **0,484** (P=2) a **0,867** (P=16), depois recua a **0,769** (P=32). Ela quase coincide com a forte em cada P; logo, ampliar o problema não eliminou a perda em P=32. Nessa configuração há **32 mil blocos** de quatro linhas e cerca de **1,02 GB** de resultados; a serialização no mestre e a transferência são causas plausíveis, mas não foram medidas separadamente.

**Tabela 3 - Tempo total (s) em P=32; RTT entre nós (ms). t/RTT usa tempo da tarefa.**

| g | med | mín | máx | RTT | t/RTT |
|---|---|---|---|---|---|
| 1 | 0.640 | 0.639 | 0.640 | 0.37 | 10.3 |
| 4 | 0.619 | 0.618 | 0.620 | 0.85 | 17.8 |
| 16 | 0.653 | 0.650 | 0.659 | 1.91 | 31.7 |
| 64 | 0.944 | 0.918 | 0.969 | 3.02 | 79.4 |
| 256 | 3.543 | 3.541 | 3.584 | 10.44 | 90.5 |

Para g=4, uma tarefa leva **15,14 ms** em média; a ida e volta da mensagem de 32.008 bytes custa **0,028 ms** no mesmo nó e **0,851 ms** entre nós: razões computação/comunicação de **536×** e **17,8×**. O RTT entre nós é cerca de 30× maior. A Tabela 3 mostra g=4 como o menor tempo mediano; g=1 aumenta pedidos e g≥64 perde balanceamento. O tempo g=4 difere só 0,3% do P=32 da Tabela 1 porque as séries são independentes.

## Balanceamento e causas

Com g=4, os 31 escravos receberam trabalho; as medianas por rank ficaram entre **108 e 154 linhas**, mas o tempo ocupado variou apenas de **0,579 a 0,589 s**. O saco dinâmico compensou linhas de custos desiguais. Em g=64, o tempo mediano sobe a **0,944 s** e há escravos com tempos ocupados muito distintos. Em g=256 existem só **16 blocos para 31 escravos**: 15 ranks ficam sem trabalho, enquanto o mais carregado ocupa ~**3,54 s**, praticamente todo o tempo do lote. Os logs grain_41968.out registram blocos, linhas e tempo ocupado de cada rank.

O aumento de g=4 para 256 reduz o número de mensagens, porém multiplica o tempo por **5,73×**; a causa dominante observável é a cauda de blocos grandes e heterogêneos, não a latência. Já o ganho de g=4 sobre g=1 é de ~**3,2%**, coerente com menos pedidos ao mestre. A diferença de RTT local/remoto sustenta um custo de rede relevante; a contribuição exata do mestre não pode ser quantificada com estes dados.

## Limites frente ao enunciado

A referência T1=**15,126 s** ficou abaixo dos 30-60 s exigidos. P=64 não foi medido: a alocação disponível limita 16 tarefas por nó (32 nos dois nós), apesar de o enunciado pedir HT até 64. O lscpu do nó 03 indica 8 núcleos físicos, não 16. O ping-pong resume 1.000 trocas por configuração, mas não possui 10 execuções independentes. Esses limites restringem a comparação direta com a configuração prescrita.

## Reprodução e uso de IA

Os logs brutos são seq_41967.out e {strong,weak,grain}_41968.out. As tabelas e gráficos são regenerados com **python3 scripts/analyze.py results 41967 41968**; o matplotlib ausente no cluster exigiu gerar os PNGs localmente. A IA utilizada foi **OpenAI Codex (baseado em GPT-6)**, para revisão do protocolo, scripts, interpretação e redação; os números foram conferidos nos logs.

**Fontes:** enunc.md, seções 4-7; código e logs no repositório indicado no cabeçalho (commit da entrega).